

Strings & Special Characters in C

A Comprehensive 12-Page Educational Guide

Basics • Escape Sequences • Header Functions • Challenges

1. Understanding C Strings

A string in C is not a separate data type like in modern languages. Instead, it is a **one-dimensional array of characters** terminated by a special character called the **Null Character (\0)**.

Mechanism:

The null character tells the compiler where the string ends. Without it, the computer wouldn't know where the text stops in memory.

```
char name[] = "Hello";
```

In memory, this takes 6 bytes: 'H', 'e', 'l', 'l', 'o', and '\0'.

2. Special Characters (Escape Sequences)

Escape sequences allow you to represent characters that are otherwise difficult or impossible to type into a string literal.

Sequence	Description
<code>\n</code>	New Line (Moves cursor to next line)
<code>\t</code>	Horizontal Tab (Large space)
<code>\0</code>	Null Character (End of string)
<code>\\</code>	Backslash character
<code>\"</code>	Double Quote character
<code>\'</code>	Single Quote character

Using Escape Sequences

How as backslash or a quote is printed within a string:

```
#include <stdio.h>
int main() {
    printf("Line 1\nLine 2");
    printf("\nShe said, \"Hello!\");
    return 0;
}
```

3. String Functions (string.h)

The C Standard Library provides a header called `<string.h>` that contains dozens of useful functions for manipulating character arrays.

Function	Purpose
<code>strlen(s)</code>	Returns actual length (excluding)
<code>strcpy(d, s)</code>	Copies source string to destination
<code>strcat(d, s)</code>	Appends source string to the end of destination
<code>strcmp(s1, s2)</code>	Compares two strings (0 if equal)

String Length & Copying

strlen() Example:

```
char s[] = "C Programming";  
printf("Length: %d", strlen(s)); // Output: 13
```

strcpy() Example:

```
char source[] = "Copy me";  
char dest[20];  
strcpy(dest, source);  
printf("%s", dest); // Output: Copy me
```

Joining & Comparing

strcat() (Concatenation):

```
char first[20] = "Hello ";  
char second[] = "World";  
strcat(first, second);  
printf("%s", first); // Output: Hello World
```

strcmp() (Comparison):

Returns 0 if strings match, non-zero otherwise.

```
if(strcmp("abc", "abc") == 0) {  
    printf("Strings are equal");  
}
```

4. Code Challenge: Character Access

Problem:

Print a string character by character using a loop without using the %s format specifier.

Solution Logic:

```
char s[] = "Manual Print";
for(int i = 0; s[i] != '\0'; i++) {
    printf("%c", s[i]);
}
```


5. Handling User Input

Inputting strings in C can be tricky because `scanf()` stops reading at the first space.

Using `gets()` vs `fgets()`:

While `gets()` is dangerous (it can overflow), `fgets()` is the professional choice:

```
char name[50];  
printf("Enter name: ");  
fgets(name, sizeof(name), stdin);  
printf("Hello %s", name);
```

6. Memory Structure of Strings

Visualizing `char str[5] = "Hi";`

Index	0	1	2	3	4
Value	'H'	'i'	'\0'	?	?

Indices 3 and 4 may contain "garbage values," but string functions stop at index 2 because of the Null Terminator.

7. Common String Pitfalls

- **Missing :** Manual character assignments often forget the terminator, causing "garbage" text.
- **Array Bounds:** Trying to copy a 20-char string into a 10-char array.
- **Direct Assignment:** `str1 = str2;` is NOT allowed in C. You must use `strcpy()`.



Module Completion

You have successfully covered the core concepts of C Strings, Special Characters, and Library Functions.